# Constraint Programming (CP(M)) 2026

Condensed Notes: Sessions up to 21 October

School of Computing Science, University of Glasgow

These notes are a condensed version of the four sets of session notes (introduction to CP and combinatorial optimisation; search and arc consistency; global constraints; other solvers) for use as an open-book reference in the invigilated quiz. Definitions, models, algorithms and worked examples are kept; figures, exercises, discussion prompts and external links have been removed.

## 1 Introduction to Constraint Programming and Combinatorial Optimisation

### 1.1 Combinatorial problems

We focus on *discrete* problems: the choices are drawn from finite and countable sets rather than from a continuous range of real values (the one place the reals appear is in linear relaxations). Many familiar computer-science problems have this form, including graph colouring, satisfiability, scheduling, and routing.

An *optimisation problem* asks for a feasible solution that is best according to some objective (the largest clique, the shortest schedule, the most valuable production plan). A corresponding *decision problem* asks whether a solution meeting a specified threshold exists. Decision versions are particularly important in complexity theory and in the study of NP-completeness.

#### *1.1.1 The clique problem*

Let $G = (V, E)$ be a finite simple undirected graph (all graphs are, unless stated otherwise). A subset $S \subseteq V$ is a *clique* if every pair of distinct vertices in $S$ is joined by an edge: $\forall u, v \in S,\ u \neq v \implies \{u, v\} \in E$.

The *decision version*, Clique:

**Instance:** an undirected graph $G = (V, E)$ and an integer $k$ with $1 \leq k \leq |V|$;
**Question:** does there exist a set $S \subseteq V$ such that $|S| \geq k$ and $S$ is a clique in $G$?

The output is either *yes* or *no*.

The *optimisation version*, Maximum Clique:

**Instance:** an undirected graph $G = (V, E)$;
**Feasible solutions:** all subsets $S \subseteq V$ that form cliques in $G$;
**Objective:** maximise $|S|$.

The optimal objective value is the *clique number* of $G$, $\omega(G) = \max\{\, |S| : S \subseteq V \text{ and } S \text{ is a clique} \,\}$.

#### *1.1.2 The graph-colouring problem*

Let $G = (V, E)$ be a graph and $k$ a positive integer. A *proper $k$-colouring* of $G$ is a function $c : V \to \{1, \ldots, k\}$ such that adjacent vertices receive different colours: $\forall \{u, v\} \in E,\ c(u) \neq c(v)$.

The *decision version*:

**Instance:** an undirected graph $G = (V, E)$ and a positive integer $k$;
**Question:** does there exist a proper $k$-colouring of $G$?

This is denoted $k$-Colourability when $k$ is fixed; 3-Colourability asks whether $G$ can be properly coloured using at most three colours.

The *optimisation version*, Minimum Graph Colouring:

**Instance:** an undirected graph $G = (V, E)$;
**Feasible solutions:** all proper colourings of $G$;
**Objective:** minimise the number of colours used.

The optimal objective value is the *chromatic number* $\chi(G) = \min\{\, k : G \text{ has a proper } k\text{-colouring} \,\}$.

#### *1.1.3 The satisfiability problem*

Let $F$ be a propositional formula over Boolean variables $X = \{x_1, \ldots, x_n\}$. A *truth assignment* is a function $\alpha : X \to \{\mathit{false}, \mathit{true}\}$; $\alpha$ *satisfies* $F$ if evaluating $F$ under $\alpha$ gives *true*.

The *decision problem* SAT:

**Instance:** a propositional formula $F$;
**Question:** does there exist a truth assignment $\alpha$ that satisfies $F$?

If $F$ is in conjunctive normal form (CNF) and every clause contains exactly three literals, the restricted problem is 3-SAT.

The optimisation variant Max-SAT, for $F = C_1 \wedge \cdots \wedge C_m$ in CNF:

**Instance:** the clauses $C_1, \ldots, C_m$ over a set of Boolean variables;
**Feasible solutions:** all truth assignments to those variables;
**Objective:** maximise the number of clauses satisfied.

Writing $[C_i(\alpha)]$ for 1 when $\alpha$ satisfies $C_i$ and 0 otherwise, $\text{MaxSAT}(F) = \max_\alpha \sum_{i=1}^{m} [C_i(\alpha)]$.

**Example.** Over $x_1, x_2, x_3$, let

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3).$$

This is a 3-CNF formula: a conjunction of clauses, each with exactly three literals. The assignment $\alpha(x_1) = \textit{true}$, $\alpha(x_2) = \textit{false}$, $\alpha(x_3) = \textit{true}$ satisfies $F$: substituting gives $C_1 = (\textit{true} \vee \textit{true} \vee \textit{true})$, $C_2 = (\textit{false} \vee \textit{false} \vee \textit{true})$, $C_3 = (\textit{true} \vee \textit{false} \vee \textit{false})$, $C_4 = (\textit{false} \vee \textit{true} \vee \textit{true})$, so every clause is *true* and $F$ is a *yes-instance* of 3-SAT.

### 1.2 Solver technologies

General-purpose approaches to discrete decision and optimisation problems include:

- constraint-programming (CP) solvers based on propagation and search;
- Boolean satisfiability (SAT) solvers;
- integer-programming (IP) and mixed-integer-programming solvers;
- linear-programming methods, including the simplex algorithm; and
- heuristic and metaheuristic methods, which may find good solutions without proving optimality.

The model used to express a problem is often closely connected to the solver technology.

#### *1.2.1 A first approach: exhaustive search*

The simplest approach to a decision problem is to list every possibility and check whether it is a solution: for a clique of size 5, check every set of five vertices; for SAT, check every truth assignment. This is infeasible in general.

One step further is *tree-based enumeration*: construct a search tree in which each level decides one choice (for maximum clique, whether a particular vertex belongs to the candidate clique). This is a correct algorithm, but the number of candidates still grows exponentially. Modern solvers improve on naive enumeration by pruning choices that cannot lead to feasible solutions, and by choosing carefully which branches to expand first. Tree search is the basis of most of the solving approaches in this course.

### 1.3 Constraint programming

Constraint programming is a framework for modelling and solving *constraint satisfaction problems* (CSPs). A CSP is represented as a triple $(X, D, C)$ where

- $X = \{x_1, \ldots, x_n\}$ is a set of variables;
- $D = \{D_1, \ldots, D_n\}$ gives a domain $D_i$ of possible values for each variable $x_i$; and
- $C$ is a set of constraints restricting which combinations of values may be assigned.

A *solution* is a complete assignment of values to variables, with $x_i \in D_i$ for every $i$, that satisfies every constraint in $C$.

**CSP model for graph colouring.** For a graph $G = (V, E)$ and $k$ available colours:

**Variables:** one variable $c_v$ for every vertex $v \in V$;
**Domains:** $c_v \in \{1, \ldots, k\}$ for every $v \in V$;
**Constraints:** $c_u \neq c_v$ for every edge $(u, v) \in E$.

This model separates the description of the problem from the method used to solve it: the same model may be handled by different CP solvers, each with its own propagation and search strategies.

**CSP view of satisfiability.** For a propositional formula $F$:

**Variables:** the propositional variables occurring in $F$;
**Domains:** $\{\textit{false}, \textit{true}\}$ for every variable;
**Constraints:** the clauses, or equivalently the requirement that $F$ evaluates to true.

**Constraints as relations.** A constraint may be represented as a pair $(S, R)$. Its *scope* $S = (x_{i_1}, \ldots, x_{i_r})$ is the tuple of variables involved, and its *relation* $R$ contains exactly the allowed tuples of values, so $R \subseteq D_{i_1} \times \cdots \times D_{i_r}$. Listing every allowed tuple is often impractical, so constraints are usually written compactly: $c_u \neq c_v$, a linear inequality, or a global constraint such as `all_different`.

**What kind of solution do we need?** Depending on the problem we may want every solution, any one feasible solution, the number of solutions, or an optimal solution according to a stated objective. This should be decided before choosing a model and a search strategy.

### 1.4 MiniZinc

MiniZinc is a high-level modelling language for constraint satisfaction and optimisation. A model describes the variables, domains, constraints and objective independently of a particular backend solver.

**Graph colouring with a data file.** The graph is described by arrays of edge endpoints; the numbers of vertices, edges and colours are supplied in a `.dzn` data file.

```
int: n;
set of int: NODE = 1..n;

int: m;
set of int: EDGE = 1..m;
array[EDGE] of NODE: from;
array[EDGE] of NODE: to;

int: k;
set of int: COLOUR = 1..k;
array[NODE] of var COLOUR: colour;

constraint forall(e in EDGE)(
    colour[from[e]] != colour[to[e]]
);

solve minimize max(colour);
```

One possible data file:

```
n = 5;
m = 6;
k = 4;
from = [1, 1, 2, 3, 3, 4];
to   = [2, 4, 3, 4, 5, 5];
```

Declarations without `var` (`n`, `m`, `k`, `from`, `to`) are parameters fixed by the data; declarations with `var` are decision variables. MiniZinc's global `all_different` constraint expresses the requirement that a collection of variables all take distinct values.

## 2 Search and Arc Consistency

A CSP is a triple $(X, D, C)$: a set of variables $X$, a domain of possible values for each variable ($D$), and a set of constraints $C$ that an acceptable solution must meet.

Running example for this section:

**Variables:** $X = \{a, b, c\}$;
**Domains:** $D(a) = D(b) = D(c) = \{1, 2, 3\}$;
**Constraints:** $C = \{C_1, C_2\}$, where $C_1 : a < b$ and $C_2 : b < c$.

### 2.1 Search

**Generate-and-test.** Generate every possible assignment within the joint domains of the variables and test whether each is a solution. Hard-coded for the example:

```
for value_a in D(a):
    for value_b in D(b):
        for value_c in D(c):
            if value_a < value_b and value_b < value_c:
                return value_a, value_b, value_c
return UNSAT
```

In the worst case this is a large number of checks; for a larger problem there are far too many to be feasible.

**Backtrack search.** Search the space by depth-first search, but give up on a branch (*prune* it and *backtrack*) when we know it cannot be part of any solution. One such situation: when all the variables involved in some constraint have been fixed and that constraint is not satisfied. In the example, a partial assignment with $a = 2$ and $b = 1$ cannot be part of any solution (since $C_1$ is violated) and can be pruned. This is still asymptotically exponential time, and will often be too slow; approaches to speed it up include successor ordering, arc consistency, and stronger consistencies.

### 2.2 Arc consistency (AC)

The idea of arc consistency is to decrease the size of the domains before we start searching; sometimes the problem is solved without any search.

**Constraint networks.** Represent the relationships between variables and constraints as a bipartite graph:

- each variable is a node (conventionally a circle);
- each constraint is a node (conventionally a rectangle);
- there is an edge (an *arc*) between a variable node and a constraint node when that variable is involved in that constraint.

**Definition.** An arc between a variable $x$ and a binary constraint $C_i(x, y)$ is **arc consistent** if for each value in the domain of $x$ there is a value in the domain of $y$ such that the pair satisfies $C_i$. The whole CSP is arc consistent if every arc is.

**Making a CSP arc consistent.** Remove from each domain every value that is not consistent. Doing so might make other arcs inconsistent, so other domains may need adjusting in turn. Not every arc needs reconsidering: if we remove a value from the domain of variable $x$, we need only reconsider arcs *from other variables* to constraints that also involve $x$.

### *2.2.1 Algorithm*

```
function ac(X, D, C):
    # Takes: variables X, domains D, constraints C
    # Gives: arc consistent domains for variables
    set D_new gets empty set
    set todo gets empty set
    for each variable x in X:
        D_new[x] gets D[x]
        todo gets all arcs from x to a constraint involving x
    while todo is not empty:
        get an arc (x, c) from todo
        remove (x, c) from todo
        y is the other variable involved in c
        domain_changed gets False
        for every value_x in D_new[x]:
            remove_me gets True
            for every value_y in D_new[y]:
                if value_x, value_y satisfy c:
                    remove_me gets False
                    stop loop
            if remove_me:
                remove value_x from D_new[x]
                domain_changed gets True
        if domain_changed:
            add all arcs from other variables to
                constraints involving x to todo
```

This is essentially the classic **AC-3** algorithm (Mackworth, 1977).

**Three possible outcomes** after running the algorithm:

1. every domain has only one value;
2. at least one domain has no values;
3. all domains have at least one value, and some have more.

### *2.2.2 Examples*

Write revise$(x, y)$ for the step "remove any value from $D(x)$ that has no supporting value in $D(y)$", i.e. processing the arc from $x$ to the constraint it shares with $y$.

**Example 1: the running integer example.** $X = \{a, b, c\}$, $D(a) = D(b) = D(c) = \{1, 2, 3\}$, $C_1 : a < b$, $C_2 : b < c$.

- revise$(a, b)$: $a = 3$ has no $b > 3$ available, so remove it. $D(a) = \{1, 2\}$. $D(a)$ changed, so re-queue revise$(b, a)$.
- revise$(b, a)$: $b = 1$ has no $a < 1$ available, so remove it. $D(b) = \{2, 3\}$. Re-queue revise$(a, b)$ and revise$(c, b)$.
- revise$(b, c)$: $b = 3$ has no $c > 3$ available, so remove it. $D(b) = \{2\}$. Re-queue revise$(a, b)$ and revise$(c, b)$.
- revise$(c, b)$: with $D(b) = \{2\}$, $c = 1$ and $c = 2$ have no supporting $b$ (need $b < c$), so remove both. $D(c) = \{3\}$. Re-queue revise$(b, c)$.
- revise$(a, b)$: with $D(b) = \{2\}$, $a = 2$ has no $b > 2$ available, so remove it. $D(a) = \{1\}$. Re-queue revise$(b, a)$.
- revise$(b, c)$, revise$(b, a)$: no further removals.

Final domains: $D(a) = \{1\}$, $D(b) = \{2\}$, $D(c) = \{3\}$: outcome (1). Arc consistency alone found the unique solution $a = 1, b = 2, c = 3$, with no search required.

**Example 2: arc consistency is not always enough.** A triangle of "not equal" constraints: $X = \{a, b, c\}$, $D(a) = D(b) = D(c) = \{1, 2\}$, $C = \{a \neq b,\ b \neq c,\ a \neq c\}$. For every arc revise$(x, y)$, each value in $D(x)$ has a supporting, different value in $D(y)$, because both domains are $\{1, 2\}$. Nothing is ever removed.

Final domains: $D(a) = D(b) = D(c) = \{1, 2\}$: outcome (3), and the CSP is already arc consistent. But there is **no solution**: three mutually-adjacent variables cannot take pairwise-different values from a domain of size 2. Arc consistency only checks pairs of variables, so it cannot see this; backtracking search would still need to try, e.g., $a = 1, b = 2$, then discover $c$ has nothing left ($c \neq a$ and $c \neq b$ rule out both values) and backtrack. Outcome (3) does not mean "solvable", only "not yet disproved by arc consistency".

**Example 3: graph colouring.** A path of three vertices $u_1 - u_2 - u_3$, where $u_1$ has already been fixed to colour 1 (perhaps by an earlier decision): $D(u_1) = \{1\}$, $D(u_2) = \{1, 2\}$, $D(u_3) = \{1, 2, 3\}$, constraints $u_1 \neq u_2$ and $u_2 \neq u_3$.

- revise$(u_2, u_1)$: $u_2 = 1$ has no supporting $u_1 \neq 1$ (since $D(u_1) = \{1\}$), so remove it. $D(u_2) = \{2\}$. Re-queue

revise$(u_3, u_2)$.
- revise$(u_1, u_2)$: no removals ($u_1 = 1$ is supported by $u_2 = 2$).
- revise$(u_3, u_2)$: with $D(u_2) = \{2\}$, $u_3 = 2$ has no supporting $u_2 \neq 2$, so remove it. $D(u_3) = \{1, 3\}$. Re-queue revise$(u_2, u_3)$.
- revise$(u_2, u_3)$: no further removals.

Final domains: $D(u_1) = \{1\}$, $D(u_2) = \{2\}$, $D(u_3) = \{1, 3\}$: outcome (3). Fixing $u_1$'s colour propagated one step down the path and shrank $u_2$'s domain, which in turn shrank $u_3$'s domain. This cascading is why arc consistency can prune far more than checking each constraint once.

#### *2.2.3 Halting and time complexity*

**Why AC halts.** We cannot remove things from domains indefinitely: if we do an iteration with no removals and `todo` is empty, we stop; every removal makes a domain smaller, and eventually domains would be empty. Equivalently, every iteration either removes something from a domain or adds nothing to `todo`; since the sum of the domain sizes is finite, there is a finite bound on the number of times arcs are added to `todo`.

**Worst-case time complexity**, for $n$ variables, maximum domain size $d$, and $c$ constraints:

- checking one arc for consistency takes $O(d^2)$;
- the number of arcs added to `todo` over the run is $O(cd)$ (each removal from a domain re-queues arcs for the constraints involving that variable);
- so the total is $O(cd^3)$.

## 3 Global Constraints

Global constraints express a constraint between a non-fixed number of variables. Their advantages are that they

- let us express constraints that occur in our models more compactly and intuitively;
- may come with special algorithms to compute them more efficiently;
- may allow for better inference/propagation (beyond basic arc consistency).

### 3.1 All-different

**$n$-Queens.**

**Instance:** a natural number $n$;
**Question:** can we place $n$ queens on an $n \times n$ chessboard so that no two are in the same row, column, or diagonal?

Expressing this without all-different is possible but awkward. With it, in MiniZinc, where `X[i]` is the column of the queen in row $i$ (so distinct rows hold by construction):

```
int: n;
array [1..n] of var 1..n: X;
include "alldifferent.mzn";
% distinct columns
constraint alldifferent(X);
% distinct diagonals upwards
constraint alldifferent([ X[i] + i - 1 | i in 1..n]);
% distinct diagonals downwards
constraint alldifferent([ X[i] - i + 1 | i in 1..n]);

solve satisfy;
```

**Inference.** Consider

$$x, y, z \in \mathbb{N}, \qquad D(x) = D(y) = \{1, 2\}, \qquad D(z) = \{1, 2, 3\}, \qquad \text{all-different}(x, y, z).$$

What can be inferred about $z$? And could the same be obtained by arc consistency if the constraints were just $x \neq y$, $x \neq z$, $y \neq z$? Reasoning about the three variables together (as in sudoku) is a powerful feature of all-different.

### 3.2 Matchings for all-different

We want to exclude domain values that are not part of any all-different value assignment, using **maximum matchings**. Visualise the possible assignments as a bipartite graph (a graph with two parts that only has edges between the parts):

- one part is the set of variables that are all-different;
- the other part is the set of possible values;
- there is an edge from variable $v$ to value $d$ if $d$ is in $D(v)$.

A *maximum matching* on such a graph is a set of edges such that no two edges share an endpoint, and there are as many edges in the set as possible.

**Claim.** If the constraint is satisfiable, then there is a matching as large as the set of all-different variables.

**Proof sketch** ($V_1$ the variables, $V_2$ the values). Take a satisfying assignment $\Phi : V_1 \to V_2$. As it is satisfying, $|\Phi| = |V_1|$ and $\Phi(v_i) \neq \Phi(v_j)$ for all distinct $v_i, v_j \in V_1$. Let $E_M = \{(v, x) : v \in V_1,\ x \in V_2,\ \Phi(v) = x\}$. Then

$|E_M| = |V_1|$; each $(v, x) \in E_M$ is an edge of the graph, since $\Phi(v) \in D(v)$; and for every pair of edges $(v, x)$ and $(u, z)$ in $E_M$ we have $v \neq u$ and $x \neq z$. So $E_M$ is a matching covering every variable.

Hence we are only interested in edges that are in *some* maximum matching. Berge's claim (with an algorithm due to Edmonds):

> An edge belongs to some but not all maximum matchings if and only if, for an arbitrary maximum matching, it belongs to either an even alternating path which begins at a free node, or an even alternating cycle.

Definitions, for a matching $M$:

- an edge in $M$ is a *matching edge*; every edge not in $M$ is *free*;
- a node is *matched* if it is incident to a matching edge, and *free* otherwise;
- an *alternating path* or *cycle* is a path or cycle whose edges are alternately matching and free.

**Complexity.** Enforcing all-different over $k$ variables whose largest domain has size $m$ costs $O(km\sqrt{k})$ to obtain the matching information, which dominates. This is much faster than a full search, so it is likely to save time overall.

### 3.3 Circuit constraint

#### *3.3.1 Definition*

Fix $n \in \mathbb{N}$ and index a set of $n$ objects (cities, tasks, locations) by $\{1, \ldots, n\}$. Introduce one variable per index, $\text{succ}[i] \in \{1, \ldots, n\}$, meaning that object $i$ is followed by object $\text{succ}[i]$. The constraint `circuit`(succ) holds exactly when succ, viewed as a function $\{1, \ldots, n\} \to \{1, \ldots, n\}$, is a single $n$-cycle: succ is a fixed-point-free permutation, and iterating succ from any index visits all $n$ indices before returning to it. This is the natural model for a travelling-salesperson-style problem: assign every location a successor, then (in the optimisation version) minimise total edge cost.

#### *3.3.2 All-different is necessary but not sufficient*

succ must be all-different: no two indices can share a successor, and no index can be its own successor. But a fixed-point-free permutation need not be a single cycle; it can decompose into several disjoint cycles.

**Example.** $n = 4$ and $\text{succ} = [2, 1, 4, 3]$. This is a fixed-point-free permutation, so all-different is satisfied, yet $1 \to 2 \to 1$ and $3 \to 4 \to 3$ are two disjoint 2-cycles rather than one 4-cycle. All-different filtering therefore cannot rule out sub-tours; an additional propagation rule is required.

#### *3.3.3 Chains and a pruning rule*

Suppose a subset of the succ variables has been fixed. The fixed edges $(i, \text{succ}[i])$ decompose into disjoint directed paths, or *chains*; an index whose variable is not yet fixed forms a chain of length one on its own. For an index $x$ on some chain, write $\text{start}(x)$ for the first index of that chain, $\text{end}(x)$ for the last, and $\text{len}(x)$ for the number of indices on the chain. Fixing $\text{succ}[i] = j$ merges the chain ending at $i$ with the chain starting at $j$; the merged chain runs from $\text{start}(i)$ to $\text{end}(j)$, with length $\text{len}(i) + \text{len}(j)$.

**Pruning rule.** If $x$ lies on a chain with $\text{start}(x) = s$, $\text{end}(x) = e$, and $\text{len}(x) < n$, remove $s$ from $D(\text{succ}[e])$: setting $\text{succ}[e] = s$ would close a cycle of length $\text{len}(x) < n$, a sub-tour. Once $\text{len}(x) = n$, the edge $\text{succ}[e] = s$ closes the unique Hamiltonian circuit and must not be pruned.

start, end and len can be maintained with a **union-find** (disjoint-set) structure with path compression, in amortised time $O(\alpha(n))$ per operation, where $\alpha$ is the inverse Ackermann function. This is essentially the `nocycle` propagator of Caseau and Laburthe (1997).

#### *3.3.4 Worked example*

Take $n = 5$ and fix $\text{succ}[1] = 3$, then $\text{succ}[3] = 5$, then $\text{succ}[5] = 2$, in that order. Two filters act throughout: the pruning rule, and all-different (an already-claimed successor value cannot be reused elsewhere). Initially $D(\text{succ}[i]) = \{1, \ldots, 5\} \setminus \{i\}$ for every $i$.

After the three decisions the chain is $1 \to 3 \to 5 \to 2$, of length $4 < 5$, with index 4 still unattached. The pruning rule removes 1 from $D(\text{succ}[2])$ (closing the chain now would give a 4-cycle missing node 4). Then

$$D(\text{succ}[2]) = \{1, 3, 4, 5\} \setminus \{1, 3, 5\} = \{4\} :$$

3 and 5 are removed by all-different (already claimed as successors of 1 and 3), and 1 by the pruning rule. So $\text{succ}[2] = 4$ is forced. This extends the chain to $1 \to 3 \to 5 \to 2 \to 4$, of length $5 = n$. Now $D(\text{succ}[4]) = \{1, 2, 3, 5\} \setminus \{2, 3, 5\} = \{1\}$, and since $\text{len} = n$ the pruning rule permits $\text{succ}[4] = 1$, completing the circuit $1 \to 3 \to 5 \to 2 \to 4 \to 1$. Propagation determines the full circuit from three initial decisions, with no search needed.

#### *3.3.5 Completeness*

Union-find keeps this filtering cheap: $O(n\,\alpha(n))$ overall. It is not, however, complete. Full generalised arc consistency for `circuit` would require deciding, for every remaining candidate value of every variable, whether it extends to some

Hamiltonian circuit consistent with the current domains. Taking domains directly from a graph's adjacency structure reduces this to deciding Hamiltonicity of that graph, which is NP-complete (Karp, 1972). So no polynomial-time filtering algorithm achieves full consistency for `circuit` unless $P = NP$. The pruning rule is sound but incomplete, and is used together with all-different's matching-based filter, which is complete for all-different on its own.

#### *3.3.6 Circuit in MiniZinc*

```
int: n;
array[1..n] of var 1..n: succ;
include "circuit.mzn";
constraint circuit(succ);

solve satisfy;
```

The all-different filtering, chain tracking and sub-tour pruning all take place inside the single `circuit(succ)` call. The optimisation version (shortest tour, i.e. classic TSP) adds costs and an objective:

```
int: n;
array[1..n, 1..n] of int: cost;
array[1..n] of var 1..n: succ;
include "circuit.mzn";
constraint circuit(succ);

solve minimize sum(i in 1..n)(cost[i, succ[i]]);
```

### 3.4 Global cardinality constraint

A generalisation of all-different: a set of variables $X$ take their values from domain $D$, and the constraint restricts the number of times that any value can be assigned to variables, within some range. A typical use is rota scheduling, with a minimum and maximum number of people on each day and night shift. As with all-different, this can be done without a global constraint, but the global constraint is easier to write and faster to propagate.

From a guards-rota example, without global cardinality:

```
% the number of guards that must be on duty in each shift of each day
array[DAYS] of var lwbDay .. upbDay: totalOnDay;
constraint forall(d in DAYS)(
    totalOnDay[d] = sum(g in GUARDS)(roster[g,d] = DAY));
array[DAYS] of var lwbNight .. upbNight: totalOnNight;
constraint forall(d in DAYS)(
    totalOnNight[d] = sum(g in GUARDS)(roster[g,d] = NGT));
```

With it:

```
% the number of guards that must be on duty in each shift of each day
array[DAYS] of var lwbDay..upbDay: totalOnDay;
array[DAYS] of var lwbNight..upbNight: totalOnNight;
constraint forall(d in DAYS)(
    global_cardinality([roster[g,d] | g in GUARDS],
        [DAY,NGT],[totalOnDay[d],totalOnNight[d]]));
```

### 3.5 Summary

Global constraints are expressive, and typically faster than rolling your own. Some, like all-different, admit filtering that is both cheap and complete; others, like circuit, only admit filtering that is cheap but incomplete (full consistency for circuit is NP-hard), which is still very much worth having, just not the whole story on its own.

## 4 Other (More Specific) Kinds of Solvers and Encodings

### 4.1 Integer programming

An *integer program* is a mathematical optimisation problem in which the variables are integers (a linear program with variables restricted to integer values).

- Often "integer programming" means integer *linear* programming (ILP), where the constraints are linear.
- If some of the variables are not integers, it is *mixed-integer* programming (MIP).
- If the integer domains are restricted to subsets of $\{0, 1\}$ it is zero-one, or binary, integer programming.

#### *4.1.1 Knapsack example*

Packing a knapsack of snacks: each snack has a cost in backpack space and a desirability, and we maximise desirability subject to space. With apples ($a$): cost 3, gain 4; lettuces ($l$): cost 7, gain 2; carrots ($c$): cost 4, gain 2, we maximise

$$4a + 2l + 2c \qquad \text{subject to} \qquad 3a + 7l + 4c \leq \text{capacity},$$

with $a, l, c$ non-negative integers.

#### *4.1.2 Graph colouring example*

Given $G = (V, E)$, colour it with a minimum number of colours, from a set of colours $C$.

**(Boolean) variables:** for each vertex $i$ and colour $j$, $c_{i,j}$ is 1 if vertex $i$ has colour $j$ and 0 otherwise; $w_j$ is 1 if colour $j$ is used and 0 otherwise.

**Objective:** $\min \sum_j w_j$.

**Constraints:**

- every vertex gets exactly one colour: $\forall i \in V,\ \sum_j c_{i,j} = 1$;
- every edge has different-coloured ends: $\forall (u, v) \in E,\ j \in C,\ c_{u,j} + c_{v,j} \leq 1$;
- a colour is recorded as used whenever a vertex takes it: $\forall i \in V,\ j \in C,\ c_{i,j} \leq w_j$.

#### *4.1.3 How ILPs are solved*

In many cases, in similar ways to CP: branch and bound is popular. There is an additional tool: **linear relaxations**. A linear relaxation of an integer program is the same program with the integrality requirement dropped, so variables may take real values.

- Solving ILPs is NP-hard; solving linear programs is polynomial-time.
- Rounding the relaxation's solution does not necessarily give a solution to the ILP; the difference between the relaxation's optimum and the ILP's optimum is the **integrality gap**.
- The solution to the relaxation nevertheless tells us about possible solutions of the ILP: the relaxation's optimum is *as good or better* than the best solution to the ILP. For a maximisation it is the same or bigger; for a minimisation it is the same or smaller.

This bound helps in pruning during search.

### 4.2 SAT solvers

Satisfiability solvers deal specifically in solving propositional-logic expressions such as 3-CNF-SAT formulas. One of the fastest CP solvers currently available, Google OR-Tools CP-SAT, is based on a SAT solver. Various solver approaches exist, and combinations often work best.

#### *4.2.1 Davis–Putnam–Logemann–Loveland (DPLL)*

DPLL is a backtracking search over truth assignments to a CNF formula. Given the current clause set: if there are no clauses left, the formula is satisfiable; if some clause is empty, this branch fails; otherwise apply the two propagation rules below as far as possible, then choose an unassigned variable, and recurse on each of its two values in turn.

- **Unit propagation.** When a clause has only one literal (a *unit clause*), that literal must be satisfied: record the assignment, remove every clause containing that literal, and remove the negation of the literal from every clause it appears in.
- **Pure literal assignment.** When a variable appears only as a positive or only as a negative literal, assign the value that satisfies that literal and remove every clause containing it.

#### *4.2.2 Randomised algorithms*

Randomised or probabilistic algorithms (e.g. Paturi–Pudlák–Saks–Zane) are among the fastest on many SAT instance families. Roughly speaking, they set variables in a random order and resolve from there.

### 4.3 Comparing methods

This is a general question about how to do science with experiments on algorithms; there is only guidance. First, use an existing benchmark set, or think carefully about how instances are generated:

- almost all instances of hard problems are easy;
- most meaningful comparisons show strengths and weaknesses of each approach, and this often depends on the instances.

Next, assuming the methods produce the same solutions so that only time is being compared:

- timings must be done on the same machine;
- there are many complications around timing, mostly beyond scope here;
- timeouts must be accounted for somehow.

A good template for a timing figure plots one method's search time against another's, per instance, on log-log axes, across several instance families, with the timeout boundary marked.